Code:

```
# ============================================================
# AI for IoT - Practical No. 2
# Title: Data Collection from IoT Sensors and Preprocessing using Python
# ============================================================

# ------------------------------------------------------------
# STEP 1: Import Required Libraries
# ------------------------------------------------------------

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler


# ------------------------------------------------------------
# STEP 2: Simulate IoT Sensor Data
# ------------------------------------------------------------

# Generate the same random values every time the program runs
np.random.seed(42)

# Generate 50 timestamps (one reading every minute)
timestamps = pd.date_range('2026-07-17 09:00', periods=50, freq='min')

# Generate random temperature readings (Average = 30°C)
temperature = np.random.normal(loc=30, scale=2, size=50)

# Generate random humidity readings (Average = 60%)
humidity = np.random.normal(loc=60, scale=5, size=50)

# Create a DataFrame to store sensor readings
sensor_data = pd.DataFrame({
    'timestamp': timestamps,
    'temperature': temperature,
    'humidity': humidity
})
```

```
# ------------------------------------------------------------
# STEP 3: Introduce Sensor Faults
# (Missing values and duplicate record)
# ------------------------------------------------------------

# Insert missing values
sensor_data.loc[5, 'temperature'] = np.nan
sensor_data.loc[12, 'humidity'] = np.nan
sensor_data.loc[20, 'temperature'] = np.nan

# Add one duplicate row
sensor_data = pd.concat([sensor_data, sensor_data.iloc[[10]]], ignore_index=True)


# ------------------------------------------------------------
# STEP 4: Display and Save Raw Sensor Data
# ------------------------------------------------------------

print("\n================================================")
print("RAW SENSOR DATA")
print("================================================")

print("\nTotal Records:", len(sensor_data))

print("\nFirst 15 Rows:")
print(sensor_data.head(15))

# Save raw sensor data
sensor_data.to_csv("sensor_data.csv", index=False)

print("\nRaw sensor data saved as 'sensor_data.csv'")
```

```
# ------------------------------------------------------------
# STEP 5: Load the Raw Data for Preprocessing
# ------------------------------------------------------------

df = pd.read_csv("sensor_data.csv")

print("\n================================================")
print("DATA INSPECTION")
print("================================================")

print("\nShape of Dataset:")
print(df.shape)

print("\nColumn Data Types:")
print(df.dtypes)

print("\nFirst 5 Rows:")
print(df.head())

print("\nMissing Values Before Cleaning:")
print(df.isnull().sum())


# ------------------------------------------------------------
# STEP 6: Handle Missing Values
# ------------------------------------------------------------

# Replace missing values with column mean
df['temperature'] = df['temperature'].fillna(df['temperature'].mean())
df['humidity'] = df['humidity'].fillna(df['humidity'].mean())

print("\nMissing Values After Cleaning:")
print(df.isnull().sum())


# ------------------------------------------------------------
# STEP 7: Remove Duplicate Records
# ------------------------------------------------------------

print("\nRows Before Removing Duplicates:", len(df))

# Remove duplicate rows
df = df.drop_duplicates().reset_index(drop=True)

print("Rows After Removing Duplicates:", len(df))
```

```
# ------------------------------------------------------------
# STEP 8: Detect and Remove Outliers (IQR Method)
# ------------------------------------------------------------

# Calculate Quartiles
Q1 = df['temperature'].quantile(0.25)
Q3 = df['temperature'].quantile(0.75)

# Calculate Interquartile Range
IQR = Q3 - Q1

# Calculate lower and upper limits
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("\nOutlier Limits")
print("Lower Limit:", lower)
print("Upper Limit:", upper)

# Remove outliers
df = df[(df['temperature'] >= lower) &
        (df['temperature'] <= upper)]

print("Rows After Outlier Removal:", len(df))


# ------------------------------------------------------------
# STEP 9: Normalize Sensor Data
# ------------------------------------------------------------

# Create Min-Max Scaler
scaler = MinMaxScaler()

# Normalize temperature and humidity columns
df[['temperature', 'humidity']] = scaler.fit_transform(
    df[['temperature', 'humidity']]
)

print("\nNormalized Data (First 5 Rows):")
print(df.head())
```

```
# ------------------------------------------------------------
# STEP 10: Visualize the Preprocessed Data
# ------------------------------------------------------------

plt.figure(figsize=(8,4))

plt.plot(df['temperature'], label='Temperature (Normalized)')
plt.plot(df['humidity'], label='Humidity (Normalized)')

plt.xlabel("Reading Index")
plt.ylabel("Normalized Value")
plt.title("Preprocessed IoT Sensor Data")
plt.legend()

plt.tight_layout()

# Save graph
plt.savefig("preprocessed_plot.png")

# Display graph
plt.show()


# ------------------------------------------------------------
# STEP 11: Save the Final Preprocessed Dataset
# ------------------------------------------------------------

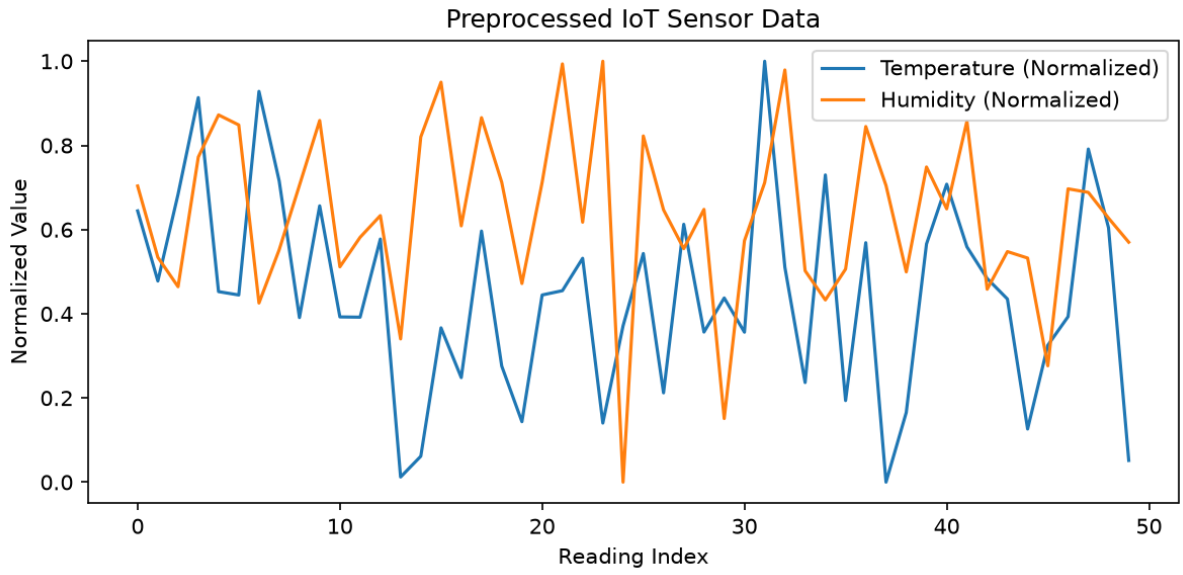
df.to_csv("preprocessed_data.csv", index=False)

print("\n==================================================")
print("PREPROCESSING COMPLETED SUCCESSFULLY")
print("==================================================")

print("\nFinal Dataset Shape:", df.shape)

print("\nFiles Generated Successfully:")
print("1. sensor_data.csv")
print("2. preprocessed_data.csv")
print("3. preprocessed_plot.png")
```

Graph:



Csv file:

```
timestamp,temperature,humidity
2026-07-17 09:00:00,30.993428306022466,61.620419846973974
2026-07-17 09:01:00,29.72347139765763,58.07458859791842
2026-07-17 09:02:00,31.295377076201383,56.61538999847021
2026-07-17 09:03:00,33.04605971281605,63.058381444204336
2026-07-17 09:04:00,29.531693250553328,65.15499761247976
2026-07-17 09:05:00,,64.65640059558099
2026-07-17 09:06:00,33.15842563101478,55.80391238388681
2026-07-17 09:07:00,31.534869458305817,58.45393812074393
2026-07-17 09:08:00,29.061051228130097,61.65631715701782
2026-07-17 09:09:00,31.08512008717193,64.8777256356118
2026-07-17 09:10:00,29.073164614375074,57.60412881077355
2026-07-17 09:11:00,29.068540492859487,59.07170511668092
2026-07-17 09:12:00,30.48392454313207,
2026-07-17 09:13:00,26.173439510684403,54.01896687959665
2026-07-17 09:14:00,26.550164334973935,64.06262911197099
2026-07-17 09:15:00,28.875424941518055,66.78120014285412
2026-07-17 09:16:00,27.974337759331153,59.63994939209833
2026-07-17 09:17:00,30.628494665190548,65.01766448946012
2026-07-17 09:18:00,28.183951848957577,61.808180125238174
2026-07-17 09:19:00,27.175392597329417,56.774401226974376
2026-07-17 09:20:00,,61.80697802754207
2026-07-17 09:21:00,29.54844739902693,67.69018283232984
2026-07-17 09:22:00,30.13505640937585,59.82086980445024
2026-07-17 09:23:00,27.150503627573087,67.82321827907003
2026-07-17 09:24:00,28.911234550949636,46.901274479551276
2026-07-17 09:25:00,30.22184517941973,64.10951252187611
2026-07-17 09:26:00,27.698012845155393,60.435235341190854
2026-07-17 09:27:00,30.751396036691343,58.50496324767066
2026-07-17 09:28:00,28.79872262016239,60.458803882677515
2026-07-17 09:29:00,29.416612500413446,50.06215542699554
2026-07-17 09:30:00,28.796586775541208,58.90164056081244
2026-07-17 09:31:00,33.70455636901787,61.78556285755873
2026-07-17 09:32:00,29.97300555052413,67.38947022370758
2026-07-17 09:33:00,27.8845781420882,57.40864890863176
2026-07-17 09:34:00,31.645089824206377,55.95753198553406
2026-07-17 09:35:00,27.558312700057954,57.491214782077314
2026-07-17 09:36:00,30.41772719000951,64.57701058851038
2026-07-17 09:37:00,26.08065975224045,61.64375554829842
2026-07-17 09:38:00,27.34362790220314,57.351198981164806
2026-07-17 09:39:00,30.393722471738247,62.56633716556678
2026-07-17 09:40:00,31.47693315999082,60.4853877467402
2026-07-17 09:41:00,30.34273656237994,64.84322495266444
2026-07-17 09:42:00,29.768703435223518,56.48973453061324
2026-07-17 09:43:00,29.397792608821423,58.36168926701116
2026-07-17 09:44:00,27.042956019265144,58.03945923433921
2026-07-17 09:45:00,28.560311583210584,52.68242525933941
2026-07-17 09:46:00,29.078722458080424,61.48060138532288
2026-07-17 09:47:00,32.11424445243783,61.30527636089945
2026-07-17 09:48:00,30.687236579136922,60.025567283212304
2026-07-17 09:49:00,26.47391968927453,58.82706433312426
2026-07-17 09:10:00,29.073164614375074,57.60412881077355
```